

SQA A-Z

The Complete Theory Guide

QA vs QC, 7 testing principles, SDLC & STLC, test levels & types, design techniques, defect life cycle, Agile & Scrum (full), metrics + 40 viva questions

Quick-scan guide for SQA interviews and vivas

Companion to: API Testing · Selenium · Playwright · JMeter · CI/CD guides

PART 1 — The foundations

1.1 QA vs QC vs Testing (asked in almost every viva)

	Quality Assurance (QA)	Quality Control (QC)	Testing
What	Improving the PROCESS so defects are prevented	Checking the PRODUCT to find defects	The activity of executing software to find defects
Focus	Prevention	Detection	Detection (a subset of QC)
Example	Defining review standards, test strategy, entry/exit criteria	Reviews, inspections, test execution	Running my Postman suite against the API

One-liner: **QA prevents, QC detects, testing executes.**

1.2 Error vs Defect/Bug vs Failure

A developer makes an **error** (human mistake) → the error lands in code as a **defect/bug** → if that code runs and misbehaves for a user, that's a **failure**. Not every defect causes a failure (dead code), and failures can also come from environment issues.

1.3 Verification vs Validation

Verification — 'Are we building the product **RIGHT**?' Checking work products against requirements/design **WITHOUT** running the software: reviews, walkthroughs, inspections (static testing).

Validation — 'Are we building the **RIGHT** product?' Running the software to check it meets the user's actual need (dynamic testing). A perfectly-built wrong product passes verification and fails validation.

1.4 The 7 Principles of Testing (ISTQB — memorize)

#	Principle	Plain meaning
1	Testing shows presence of defects, not absence	Tests can prove bugs exist, never that none remain
2	Exhaustive testing is impossible	You can't test every input — use risk & techniques to choose
3	Early testing (shift-left)	Bugs found in requirements cost pennies; in production, fortunes
4	Defect clustering	80% of bugs live in 20% of modules — focus there
5	Pesticide paradox	Same tests re-run stop finding new bugs — refresh and vary them
6	Testing is context dependent	You test a banking app differently from a game
7	Absence-of-errors fallacy	Zero bugs means nothing if the product solves the wrong problem

PART 2 — SDLC (how software is built)

SDLC = Software Development Life Cycle, the phases of building software: **Requirements** → **Design** → **Development** → **Testing** → **Deployment** → **Maintenance**. The models differ in HOW they arrange these phases.

Model	How it works	Good / Bad
Waterfall	Phases strictly one after another; next starts when previous finishes	Simple, documented / rigid — testing comes late, changes are painful

Model	How it works	Good / Bad
V-Model	Waterfall bent into a V: every dev phase pairs with a test phase (requirements↔UAT, design↔integration, code↔unit)	Testing planned early / still rigid
Iterative / Incremental	Build a small piece, test it, add the next piece	Early working software / needs good planning
Spiral	Iterations driven by RISK analysis each cycle	Great for big risky projects / heavy, costly
Agile	Small cross-functional teams deliver working software every 1–4 week sprint; welcomes change	Fast feedback, flexible / needs discipline & customer involvement

Viva favourite: "Where does testing happen in each model?" Waterfall: at the end (risky). V-Model: planned alongside each phase. Agile: continuously, inside every sprint.

PART 3 — STLC (how testing is organized)

STLC = Software Testing Life Cycle — the phases of the TESTING work itself. Each phase has entry criteria (what must exist to start) and exit criteria (what must be done to finish).

Phase	What you do	Main deliverable
1. Requirement analysis	Study requirements, find testable items, ask questions	RTM started, clarification list
2. Test planning	Scope, approach, resources, schedule, risks	Test Plan (like my TEST-PLAN.md)
3. Test case development	Write test cases & scripts, prepare test data	Test cases, automation scripts, data
4. Environment setup	Prepare servers, tools, access, data	Ready environment + smoke check
5. Test execution	Run tests, log results, report defects	Execution log, bug reports
6. Closure	Evaluate exit criteria, metrics, lessons learned	Test summary report

RTM (Requirement Traceability Matrix): a table mapping every requirement → its test cases, proving nothing is untested. My repo does this: 16 test cases each mapped to an automated request.

PART 4 — Test levels & test types

4.1 The four levels (WHO tests WHAT, WHEN)

Level	What is tested	Usually by
Unit testing	One function/class in isolation	Developers
Integration testing	Modules talking to each other (API ↔ DB, service ↔ service)	Developers/QA — my API tests live here
System testing	The complete application end-to-end against requirements	QA team
Acceptance (UAT)	Real business scenarios — 'is this what we asked for?'	End users / customer / product owner

Integration approaches: **big-bang** (join everything at once — hard to debug), **top-down** (use stubs for missing lower modules), **bottom-up** (use drivers for missing upper modules), **sandwich** (both).

4.2 Functional vs Non-functional

Functional = does it do the right THING (login works, booking created). **Non-functional** = how WELL it does it: performance, security, usability, reliability, compatibility, accessibility.

4.3 The types everyone confuses (viva gold)

Pair	The difference
Smoke vs Sanity	Smoke: broad-and-shallow 'is the build stable enough to test at all?' on a NEW build. Sanity: narrow-and-deep quick check that a SPECIFIC fix/feature works before deeper testing.
Retesting vs Regression	Retesting: re-run the FAILED test to confirm the bug fix works. Regression: re-run PASSING tests to confirm the fix broke nothing else. My CI suite = automated regression.
Black vs White vs Grey box	Black: no code knowledge, test via inputs/outputs (most QA). White: test the code internals, paths, coverage (devs). Grey: partial knowledge — e.g. I read API docs/schema but not source code.
Static vs Dynamic	Static: examine without executing (reviews, walkthroughs, lint). Dynamic: execute and observe (all test running).
Alpha vs Beta	Alpha: internal users test at the developer's site. Beta: real external users test in the real world before launch.
Exploratory vs Ad-hoc	Exploratory: unscripted but STRUCTURED — learn, design, and execute simultaneously, often time-boxed with a charter, documented. Ad-hoc: completely informal random attack, no documentation.
Positive vs Negative	Valid input expecting success vs invalid input expecting the RIGHT failure (my malformed-JSON test).

4.4 Other types to name-drop

Usability (easy to use?), **compatibility** (browsers/devices/OS), **security** (auth, injection, access control — my 403 tests), **performance** (see JMeter guide), **localization/i18n** (languages, formats), **accessibility** (usable with screen readers, WCAG), **recovery** (crash and come back?), **end-to-end** (whole user journey across systems), **monkey testing** (random inputs).

PART 5 — Test design techniques (with mini examples)

Technique	Idea	Mini example
Equivalence Partitioning	Split inputs into classes that behave the same; test one value per class	Age field 18–60: classes = <18, 18–60, >60 → test 10, 35, 70
Boundary Value Analysis	Bugs live at the edges — test exactly on and beside boundaries	For 18–60 test 17, 18, 19 and 59, 60, 61
Decision Table	Combinations of conditions → expected actions in a grid	Login: valid user × valid pass = success; the other 3 combos = specific errors
State Transition	Test valid & invalid moves between states	Booking: created → updated → deleted; deleted → GET must be 404 (my suite!)
Use Case Testing	Test complete user scenarios start-to-finish	Search hotel → book → pay → receive confirmation
Error Guessing	Attack with experience: empty, huge, special chars, wrong format	SQL quotes in name field, 0-byte file upload, malformed JSON

Technique	Idea	Mini example
Pairwise / Orthogonal	Cover all PAIRS of parameter values instead of all combos	3 browsers × 3 OS × 3 languages = 27 combos → ~9 pairwise tests

PART 6 — Test artifacts & the defect life cycle

6.1 Documents you produce

Artifact	What it is
Test strategy	Organization-level: overall approach to testing (rarely changes)
Test plan	Project-level: scope, approach, resources, schedule, risks, entry/exit criteria
Test scenario	A high-level 'what to test': 'Verify user can create a booking'
Test case	Detailed 'how': ID, preconditions, steps, test data, expected result, priority
RTM	Requirement → test case mapping proving coverage
Bug report	ID, title, environment, steps to reproduce, expected vs actual, severity, priority, evidence
Test summary report	End-of-cycle results: executed/passed/failed, open defects, coverage, go/no-go recommendation

6.2 Defect life cycle (bug's journey)

New → (triage) → **Assigned** → **Open/In progress** → **Fixed** → **Retest** → **Verified** → **Closed**. Side exits: **Rejected** (not a bug), **Duplicate** (already reported), **Deferred** (real, fix later), **Reopened** (fix didn't work — back to Assigned).

6.3 Severity vs Priority (with the 4 corners)

Combination	Example
High severity + High priority	Payment crashes on checkout — fix NOW
High severity + Low priority	App crashes in a feature 0.1% of users touch, next release is fine
Low severity + High priority	Company name misspelled on the homepage — trivial technically, fix today
Low severity + Low priority	Slight button misalignment on an admin page

Severity = technical damage (QA sets it). Priority = business urgency (product/lead sets it).

PART 7 — Agile & Scrum (full)

7.1 Agile in one breath

Agile is a mindset from the Agile Manifesto (2001) valuing: **individuals & interactions** over processes, **working software** over exhaustive documentation, **customer collaboration** over contract negotiation, and **responding to change** over following a plan. Deliver small working slices frequently; feedback drives the next slice. Scrum is the most popular FRAMEWORK for doing Agile; Kanban (continuous flow, WIP limits, no sprints) is another.

7.2 Scrum — the complete picture

	Name	What it is
Roles (3)	Product Owner	Owns WHAT to build: manages & prioritizes the product backlog, represents the customer
	Scrum Master	Owns the PROCESS: removes blockers, coaches the team, protects it from interruptions — a servant leader, NOT a boss
	Development Team	Cross-functional doers — developers AND testers together, typically 3–9 people, self-organizing
Events (5)	Sprint	The container: a fixed 1–4 week cycle producing a potentially shippable increment
	Sprint Planning	Team selects backlog items for the sprint and plans how
	Daily Scrum (stand-up)	15 min, same time daily: what I did / will do / what blocks me
	Sprint Review	Demo the increment to stakeholders, gather feedback
	Sprint Retrospective	Team-only: what went well, what didn't, what we'll improve
Artifacts (3)	Product Backlog	Prioritized wish-list of everything the product might need
	Sprint Backlog	The items chosen for THIS sprint + the plan
	Increment	The working, tested software produced by the sprint

7.3 Scrum vocabulary you must know

Term	Meaning
User story	A requirement in user language: 'As a , I want , so that '
Acceptance criteria	The testable conditions that make a story DONE — QA's best friend; I derive test cases straight from them
Story points	Relative effort estimate (Fibonacci: 1,2,3,5,8...) — complexity, not hours
Velocity	Story points a team completes per sprint — used to forecast, never to compare teams
Definition of Done (DoD)	Team-wide checklist for ANY item: coded, reviewed, tested, automated, documented
Definition of Ready	Checklist a story must meet BEFORE entering a sprint (clear, estimated, has acceptance criteria)
Epic	A story too big to fit a sprint — split into smaller stories
Backlog grooming / refinement	Ongoing meeting to clarify, split, and estimate upcoming backlog items
Burndown chart	Remaining work vs time in the sprint — is the team on track?
Spike	A time-boxed research task to answer a technical question

7.4 QA's role in Scrum (your answer to 'how do you work in Agile?')

"Testers are inside the team, not after it. I join refinement and challenge acceptance criteria (shift-left — preventing bugs before code exists), write test cases while devs write code, test stories the moment they're dev-complete inside the SAME sprint, automate regression so it runs in CI on every commit, report in stand-ups, and demo quality status at the review. Testing isn't a phase — it's a continuous activity in every sprint."

PART 8 — Metrics & manual-vs-automation

Common QA metrics

Metric	Formula / meaning
Test execution %	Executed / total planned tests
Pass rate	Passed / executed
Defect density	Defects per module or per KLOC — reveals weak areas (defect clustering!)
Defect leakage	Bugs found in PRODUCTION / total bugs — the metric QA fears; measures what escaped
Defect rejection ratio	Rejected bug reports / total reported — measures QA report quality
Test coverage	Requirements (or code) covered by tests / total
MTTD / MTTR	Mean time to detect / repair a defect

What to automate vs keep manual

Automate: regression, smoke, data-driven repetition, API contracts, anything run often and stable. **Keep manual:** exploratory testing, usability judgment, one-off checks, wildly unstable UIs. Rule: automation checks what we already know; humans explore what we don't. ROI question: automation pays back when (runs × manual time saved) exceeds (build + maintain cost).

PART 9 — 40 Viva Questions with Answers

A. Fundamentals

Q: What is software testing?

A: Evaluating software by executing it to find defects and to check it meets requirements — providing information about quality and risk.

Q: QA vs QC vs testing?

A: QA improves the process to PREVENT defects; QC examines the product to DETECT them; testing is the execution activity within QC.

Q: Error vs defect vs failure?

A: Human mistake (error) → flaw in code (defect/bug) → wrong behavior at runtime (failure).

Q: Verification vs validation?

A: Verification: building the product right — reviews without executing. Validation: building the right product — executing and checking against user needs.

Q: Name the 7 testing principles.

A: Shows presence not absence; exhaustive testing impossible; early testing; defect clustering; pesticide paradox; context dependent; absence-of-errors fallacy.

Q: What is the pesticide paradox?

A: Re-running the same tests stops finding new bugs — like insects growing resistant. Regularly review and add new/varied tests.

Q: Why is exhaustive testing impossible?

A: Even one 10-character text field has astronomically many inputs. We use risk-based prioritization and design techniques to pick the strongest small set.

Q: What is shift-left?

A: Moving testing earlier in the lifecycle — reviewing requirements, testing per-commit in CI — because bugs get exponentially costlier the later they're found.

Q: What is a test oracle?

A: The source that tells you the EXPECTED result — requirements, specs, docs, or a comparable system.

Q: What is risk-based testing?

A: Prioritizing tests by likelihood × impact of failure — payment flow gets more testing than the about-us page.

B. SDLC / STLC / Levels & Types

Q: What is SDLC? Name models.

A: Software Development Life Cycle: requirements → design → build → test → deploy → maintain.
Models: Waterfall, V-Model, Iterative, Spiral, Agile.

Q: Waterfall vs Agile?

A: Waterfall: strict sequential phases, testing at the end, change is expensive. Agile: short sprints delivering working software, testing continuous, change welcomed.

Q: What is the V-Model?

A: Every development phase pairs with a testing phase — requirements↔UAT, design↔system/integration, coding↔unit — so test planning starts early.

Q: What are the STLC phases?

A: Requirement analysis → test planning → test case development → environment setup → execution → closure; each with entry and exit criteria.

Q: What are entry and exit criteria?

A: Conditions to START a phase (e.g., environment ready, build deployed) and to FINISH it (e.g., 95% pass rate, no open critical bugs). Mine are in TEST-PLAN.md.

Q: What is an RTM?

A: Requirement Traceability Matrix — maps each requirement to its test cases, proving complete coverage and showing the impact of requirement changes.

Q: Name the 4 test levels.

A: Unit (one function, by devs) → integration (modules together — my API tests) → system (whole app) → acceptance/UAT (business validation by users).

Q: Smoke vs sanity?

A: Smoke: broad-shallow build-stability check on a new build. Sanity: narrow-deep check of a specific fix before deeper testing.

Q: Retesting vs regression?

A: Retesting confirms the FIXED bug is gone (re-run the failed case). Regression confirms the fix broke nothing ELSE (re-run the passing suite). My CI runs regression on every push.

Q: Black box vs white box vs grey box?

A: Black: behavior only, no code view. White: code internals, paths, coverage. Grey: partial knowledge — like testing an API from its docs and schema without the source.

Q: What is exploratory testing?

A: Simultaneous learning, test design, and execution — structured (charters, time-boxes, notes) but unscripted. Finds bugs scripts miss.

Q: What is UAT?

A: User Acceptance Testing — end users/product owner run real business scenarios to confirm the software fits the need before go-live.

Q: What is end-to-end testing?

A: Validating a complete user journey across all integrated systems — e.g., search → book → pay → email confirmation.

Q: Functional vs non-functional?

A: Functional: does it do the right thing. Non-functional: how well — performance, security, usability, reliability, compatibility.

C. Techniques, Defects & Documents

Q: Explain equivalence partitioning with an example.

A: Group inputs into classes behaving the same and test one per class. Age 18–60: test one value below, one inside, one above — 3 tests instead of hundreds.

Q: Explain boundary value analysis.

A: Defects cluster at edges, so test on and around boundaries: for 18–60 → 17,18,19 and 59,60,61. I applied it with price 1 and 9999 and a huge non-existent ID.

Q: What is a decision table?

A: A grid of condition combinations → expected actions; guarantees you cover every combination of business rules (e.g., 4 login combos).

Q: What is state transition testing?

A: Testing valid and invalid moves between states. My booking lifecycle is exactly this: create → update → delete → then GET must 404.

Q: Test scenario vs test case?

A: Scenario: high-level what ('verify booking creation'). Test case: detailed how — steps, data, expected result.

Q: What makes a GOOD test case?

A: Atomic (one thing), clear steps anyone can follow, explicit expected result, independent of other cases, traceable to a requirement.

Q: What goes in a bug report?

A: ID, precise title, environment, steps to reproduce, expected vs actual, severity+priority, evidence (screenshots/logs/request-response). Mine follow docs/BUG-REPORT-TEMPLATE.md.

Q: Explain the defect life cycle.

A: New → Assigned → Open → Fixed → Retest → Verified → Closed; with Rejected, Duplicate, Deferred, and Reopened as side paths.

Q: Severity vs priority with an example of low-severity/high-priority?

A: Severity = technical damage; priority = business urgency. Company name misspelled on the homepage: trivially low severity, but fix-today priority.

Q: What if a developer rejects your bug?

A: Stay factual: reproduce it again, attach evidence (steps, video, logs, exact build), check the requirement/acceptance criteria, and discuss. If it's genuinely as-designed, I accept and document; if we disagree, the product owner decides.

Q: What if you find a critical bug just before release?

A: Report immediately with evidence and impact analysis; the go/no-go is a business decision — my job is to make the risk crystal clear to the decision makers, not to hide or inflate it.

Q: What is defect leakage?

A: Bugs that escaped to production ÷ total bugs — measures how much the QA net missed; drives root-cause analysis and new regression tests.

D. Agile & Scrum

Q: What is Agile?

A: An iterative mindset: deliver small working increments every sprint, get feedback, welcome change — from the Agile Manifesto's four values.

Q: What is Scrum? Its 3-5-3?

A: The most popular Agile framework: 3 roles (Product Owner, Scrum Master, Dev Team), 5 events (Sprint, Planning, Daily Scrum, Review, Retrospective), 3 artifacts (Product Backlog, Sprint Backlog, Increment).

Q: Product Owner vs Scrum Master?

A: PO owns WHAT: the backlog, priorities, customer voice. SM owns the PROCESS: removes blockers, coaches, protects the team — servant leader, not manager.

Q: What happens in the daily stand-up?

A: 15 minutes, everyone answers: what I did yesterday, what I'll do today, what's blocking me. Sync, not status-report-to-boss.

Q: Sprint review vs retrospective?

A: Review: demo the increment to stakeholders — about the PRODUCT. Retrospective: team-only improvement talk — about the PROCESS.

Q: What is a user story and acceptance criteria?

A: 'As a I want so that ' plus testable conditions defining done. I derive my test cases directly from acceptance criteria.

Q: What are story points and velocity?

A: Relative effort estimates (Fibonacci) and the points a team finishes per sprint — for forecasting, never for comparing teams or people.

Q: What is the Definition of Done?

A: A team checklist every item must satisfy: coded, reviewed, unit-tested, QA-tested, automated, documented. Testing is INSIDE done, not after it.

Q: How does QA work inside a sprint?

A: Join refinement to challenge acceptance criteria, write cases while devs code, test within the sprint, automate regression into CI, raise quality risks in stand-ups, support the demo.

Q: Scrum vs Kanban?

A: Scrum: fixed sprints, roles, and ceremonies. Kanban: continuous flow, a board with WIP limits, no fixed iterations. Both are Agile.

Last-minute quick scan (the night-before page)

If they say...	You say...
QA/QC/testing?	Prevent process / detect product / execute.
V&V;?	Building it right (reviews) / building the right it (execution).
7 principles?	Presence-not-absence, no exhaustive, early, clustering, pesticide, context, no-error-fallacy.
SDLC models?	Waterfall, V, iterative, spiral, Agile.
STLC?	Analyze → plan → design → environment → execute → close.
4 levels?	Unit → integration → system → UAT.
Smoke/sanity?	Broad-shallow new build / narrow-deep specific fix.
Retest/regression?	Fix confirmed / nothing-else-broke.
Techniques?	EP, BVA, decision table, state transition, error guessing, pairwise.
Defect cycle?	New→Assigned→Fixed→Retest→Verified→Closed (+rejected/duplicate/deferred/reopened).
Severity/priority?	Technical damage vs business urgency — homepage typo = low/high.
Scrum 3-5-3?	PO+SM+Team / Sprint+Planning+Daily+Review+Retro / ProductBL+SprintBL+Increment.
DoD?	Team checklist — tested is part of done.
Your project?	Postman/Newman API suite: 16 cases, 41 assertions, CRUD+negative+security+schema, CI on GitHub Actions, 2 real defects documented.

Golden rule for the whole viva: definition + tiny example + "in my project I...". Theory proves you studied; the project proves you can DO it.